

Email Integrity and Tamper-Proofing Strategy

Digital Signatures and Hashing Algorithms

Deliverable 3 of 6

Table of Contents

1. Introduction
2. Why Digital Signatures Matter
3. GPG Implementation
4. Tamper Detection Demonstration
5. Hashing Algorithms
6. Signature Workflow
7. Recommendations
8. Conclusion

1. Introduction

Digital signatures are the most direct technical mechanism to guarantee email integrity and prove sender identity. This deliverable explains how I implemented them using GPG, demonstrated tamper detection in practice, and how the same principles apply to corporate email at scale.

2. Why Digital Signatures Matter

Email travels through multiple servers, networks, and intermediaries before reaching the recipient. At any of these hops, the content could be modified - either accidentally through encoding issues or deliberately by an attacker. A digital signature provides two things that no other control delivers together:

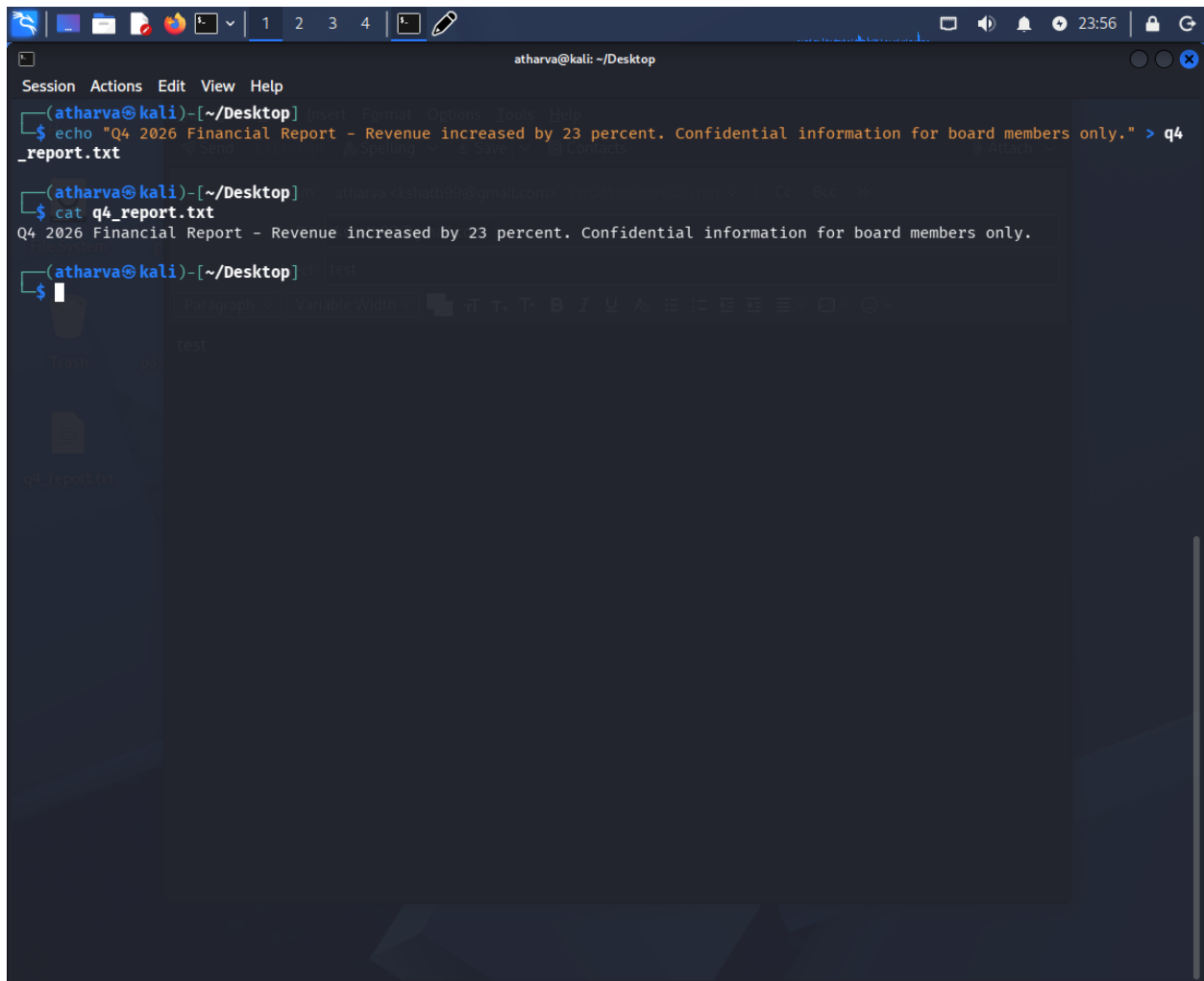
- Integrity - any change to the message content, however small, invalidates the signature
- Authentication - only the holder of the private key could have produced a valid signature for that message

Together these properties also give non-repudiation, meaning the sender cannot later deny having sent the signed message. This is legally significant in many jurisdictions including the Indian IT Act 2000.

3. GPG Implementation

3.1 Creating the Document

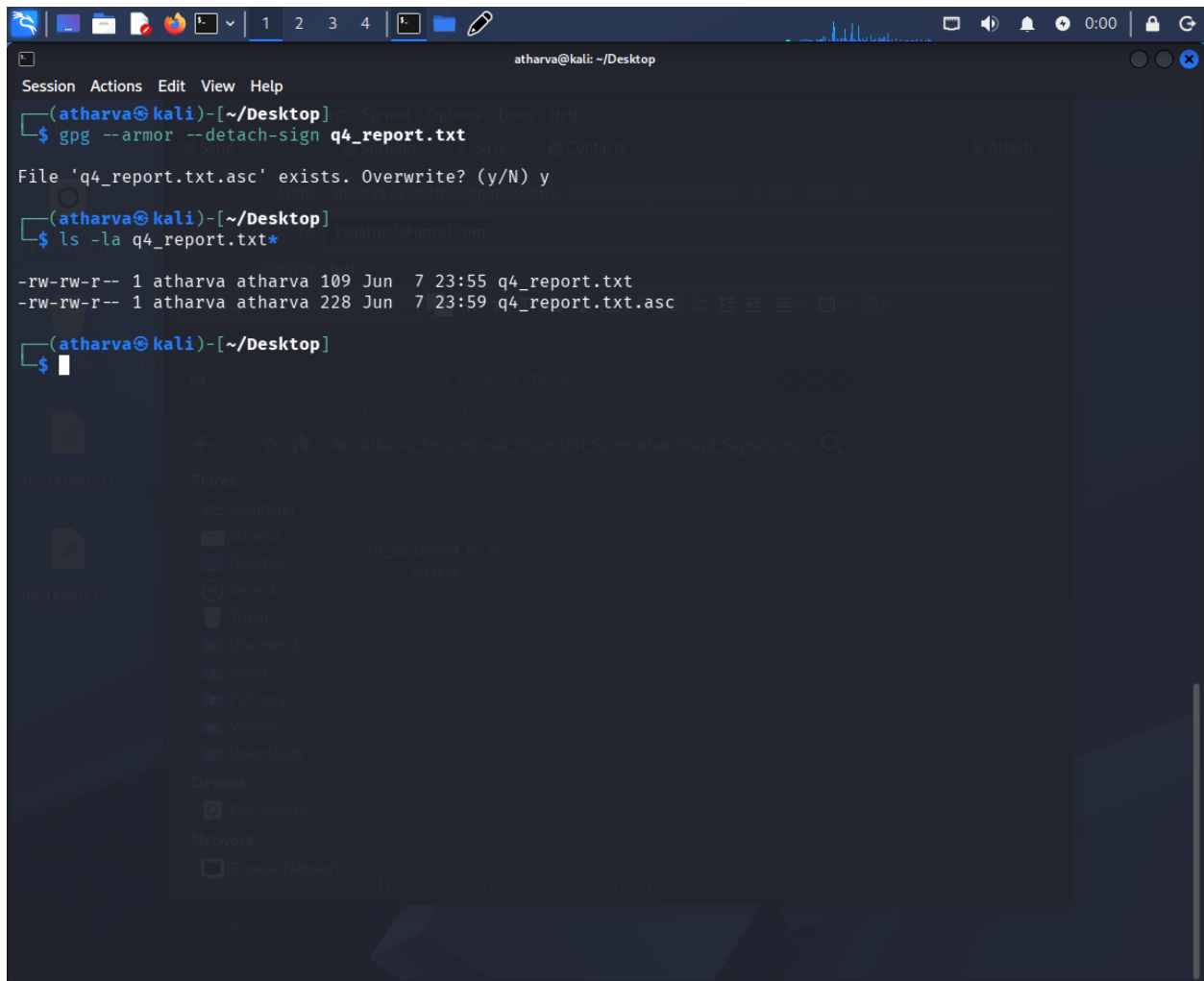
A sample financial report was created to test the signing workflow. This represents the kind of internal document that a consulting firm would routinely sign before distribution.

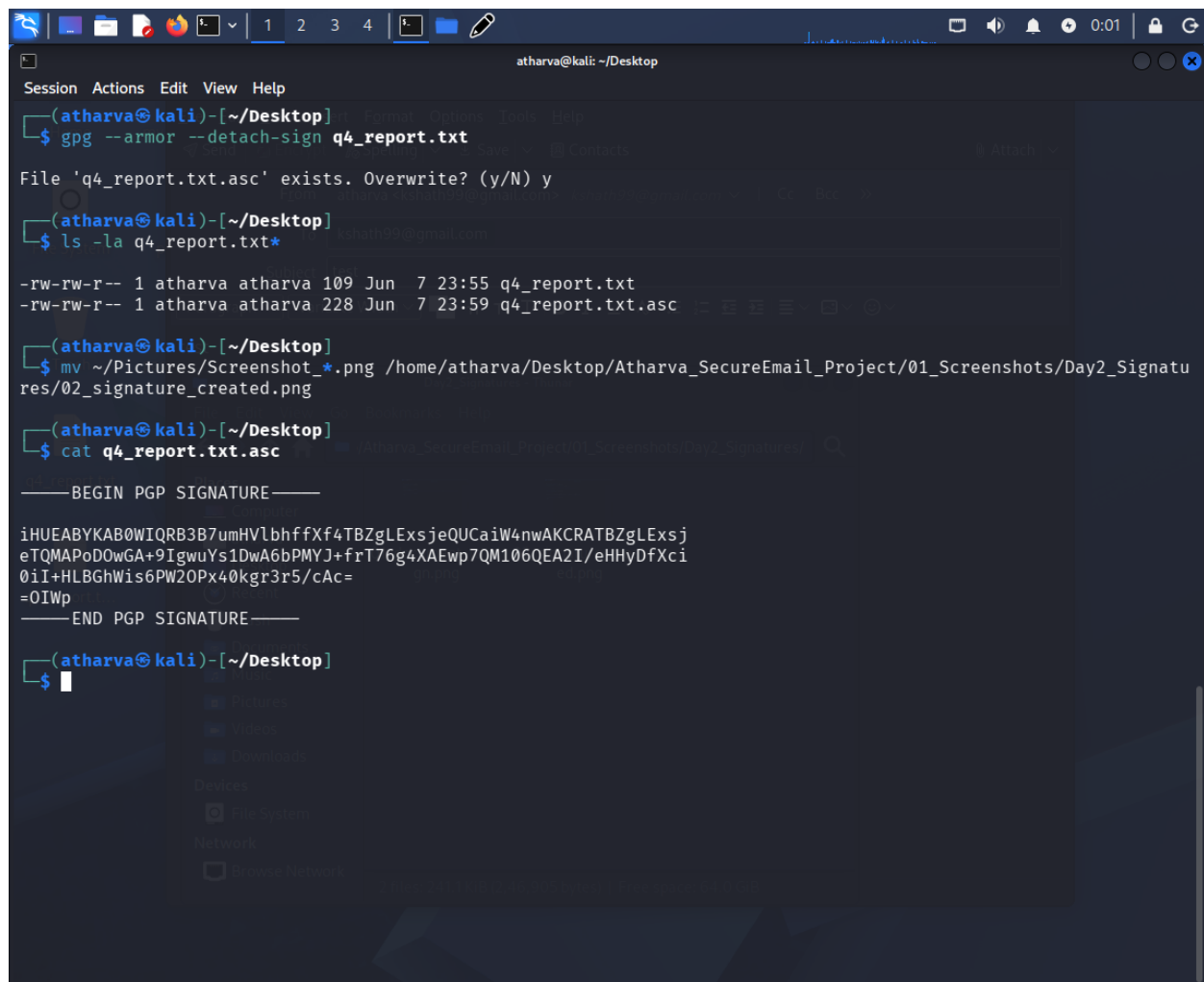


```
atharva@kali: ~/Desktop
Session Actions Edit View Help
(atharva@kali)-[~/Desktop]
$ echo "Q4 2026 Financial Report - Revenue increased by 23 percent. Confidential information for board members only." > q4_report.txt
(atharva@kali)-[~/Desktop]
$ cat q4_report.txt
Q4 2026 Financial Report - Revenue increased by 23 percent. Confidential information for board members only.
(atharva@kali)-[~/Desktop]
$
```

3.2 Generating the Signature

A detached signature was created using `gpg --armor --detach-sign`. A detached signature is stored in a separate .asc file, leaving the original document untouched - this is the format used by most enterprise email clients.

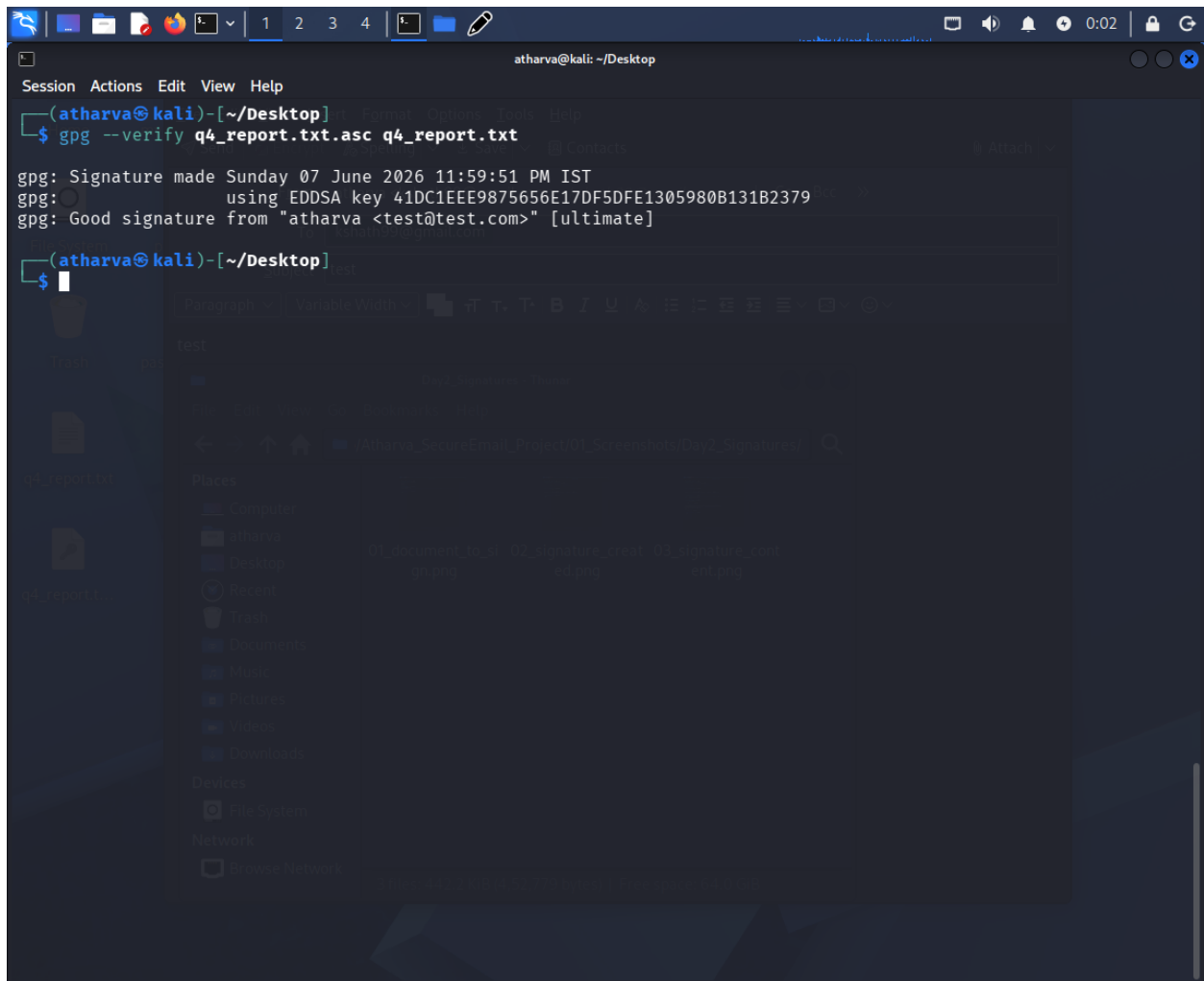




```
atharva@kali: ~/Desktop
Session Actions Edit View Help
(atharva@kali)-[~/Desktop]
$ gpg --armor --detach-sign q4_report.txt
File 'q4_report.txt.asc' exists. Overwrite? (y/N) y
(atharva@kali)-[~/Desktop]
$ ls -la q4_report.txt*
-rw-rw-r-- 1 atharva atharva 109 Jun  7 23:55 q4_report.txt
-rw-rw-r-- 1 atharva atharva 228 Jun  7 23:59 q4_report.txt.asc
(atharva@kali)-[~/Desktop]
$ mv ~/Pictures/Screenshot_*.png /home/atharva/Desktop/Atharva_SecureEmail_Project/01_Screenshots/Day2_Signatures/02_signature_created.png
(atharva@kali)-[~/Desktop]
$ cat q4_report.txt.asc
-----BEGIN PGP SIGNATURE-----
iHUEABYKAB0WIQRB3B7umHVlbhffXf4TBZgLExsjeQUCaiW4nwAKCRATBZgLExsj
eTQMAPoD0wGA+9IgwuYs1DwA6bPMYJ+frT76g4XAEwp7QM106QEA2I/eHHyDfXci
0iI+HLBGhWis6PW20Px40kgr3r5/cAc=
=OIWp
-----END PGP SIGNATURE-----
(atharva@kali)-[~/Desktop]
$
```

3.3 Verifying the Signature

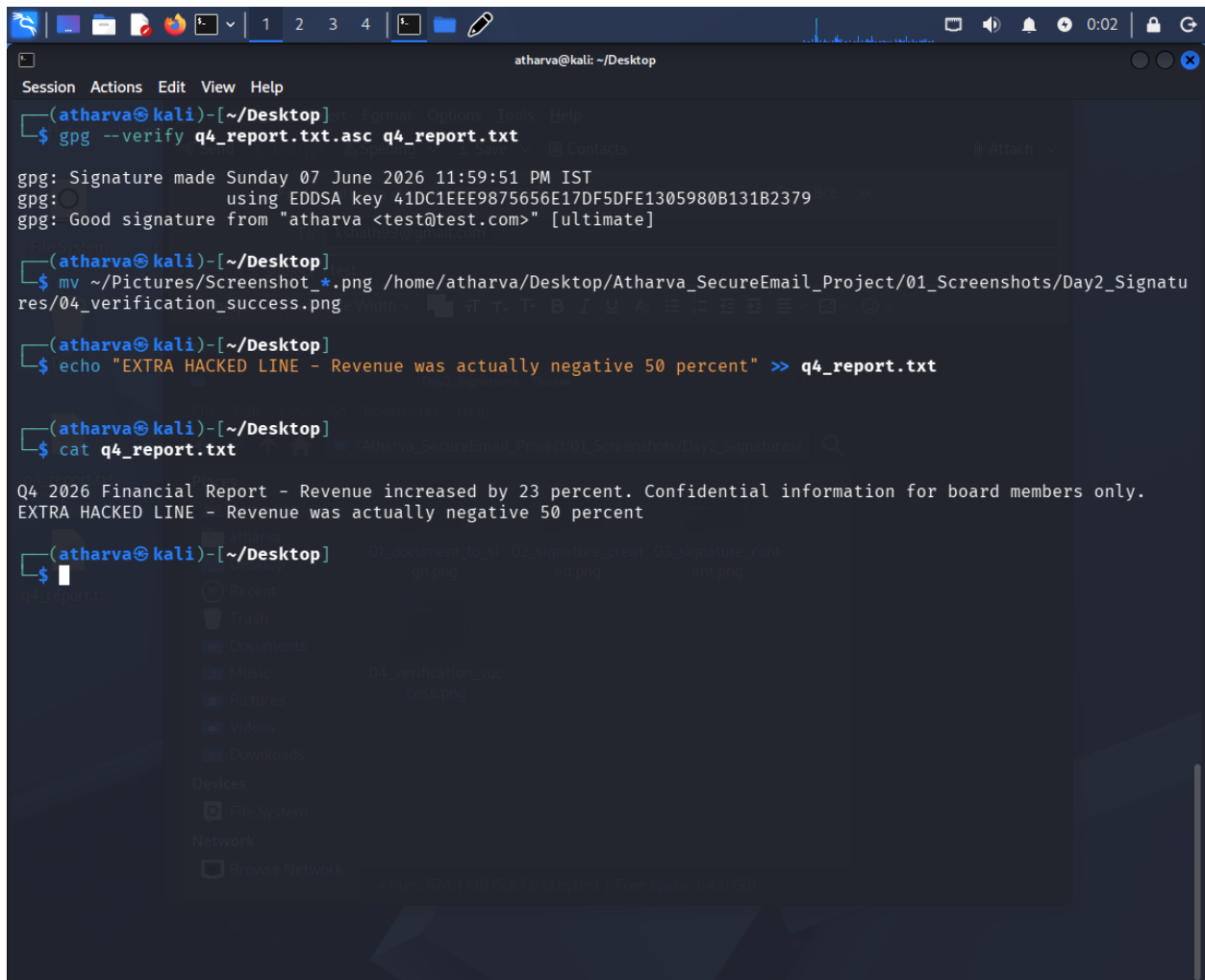
The verification command confirms three things at once: the signature is valid, it was made by the holder of the matching private key, and the document content has not changed since signing.



4. Tamper Detection Demonstration

This is the strongest practical proof in the deliverable. After confirming the original signature verified correctly, I deliberately tampered with the document by appending a fake line claiming revenue was negative. The same verification command then failed loudly with a 'BAD signature' message.

4.1 Tampering the File



```
atharva@kali: ~/Desktop
Session Actions Edit View Help
(atharva@kali)~[~/Desktop]
$ gpg --verify q4_report.txt.asc q4_report.txt

gpg: Signature made Sunday 07 June 2026 11:59:51 PM IST
gpg: using EDDSA key 41DC1EEE9875656E17DF5DFE1305980B131B2379
gpg: Good signature from "atharva <test@test.com>" [ultimate]

(atharva@kali)~[~/Desktop]
$ mv ~/Pictures/Screenshot_*.png /home/atharva/Desktop/Atharva_SecureEmail_Project/01_Screenshots/Day2_Signatures/04_verification_success.png

(atharva@kali)~[~/Desktop]
$ echo "EXTRA HACKED LINE - Revenue was actually negative 50 percent" >> q4_report.txt

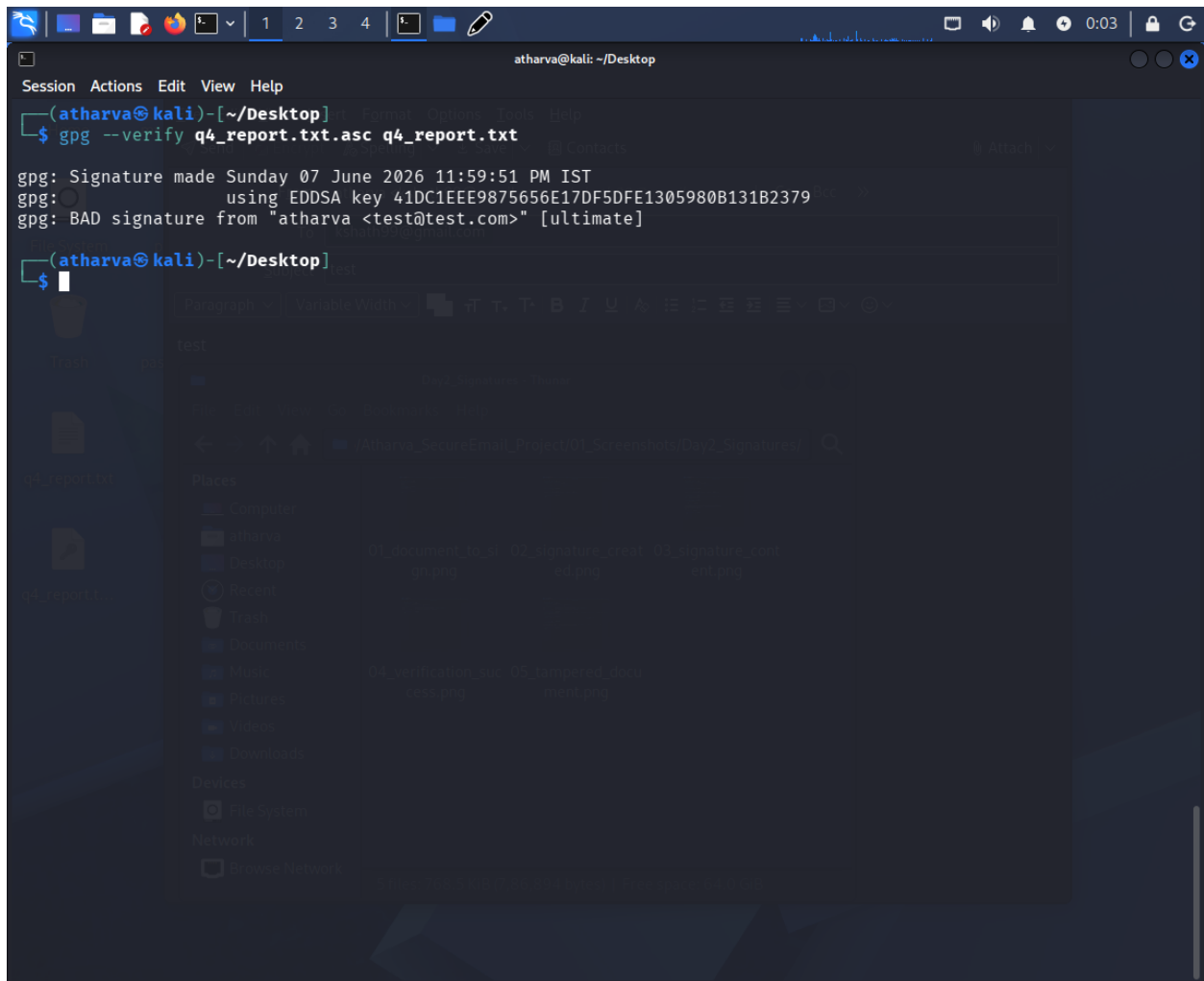
(atharva@kali)~[~/Desktop]
$ cat q4_report.txt

Q4 2026 Financial Report - Revenue increased by 23 percent. Confidential information for board members only.
EXTRA HACKED LINE - Revenue was actually negative 50 percent

(atharva@kali)~[~/Desktop]
$
```

4.2 Failed Verification

GPG immediately reports the signature mismatch. In a production mail client this would block delivery or display a clear warning to the user.



This proves the system detects any change, no matter how small. Even a single character change invalidates the signature.

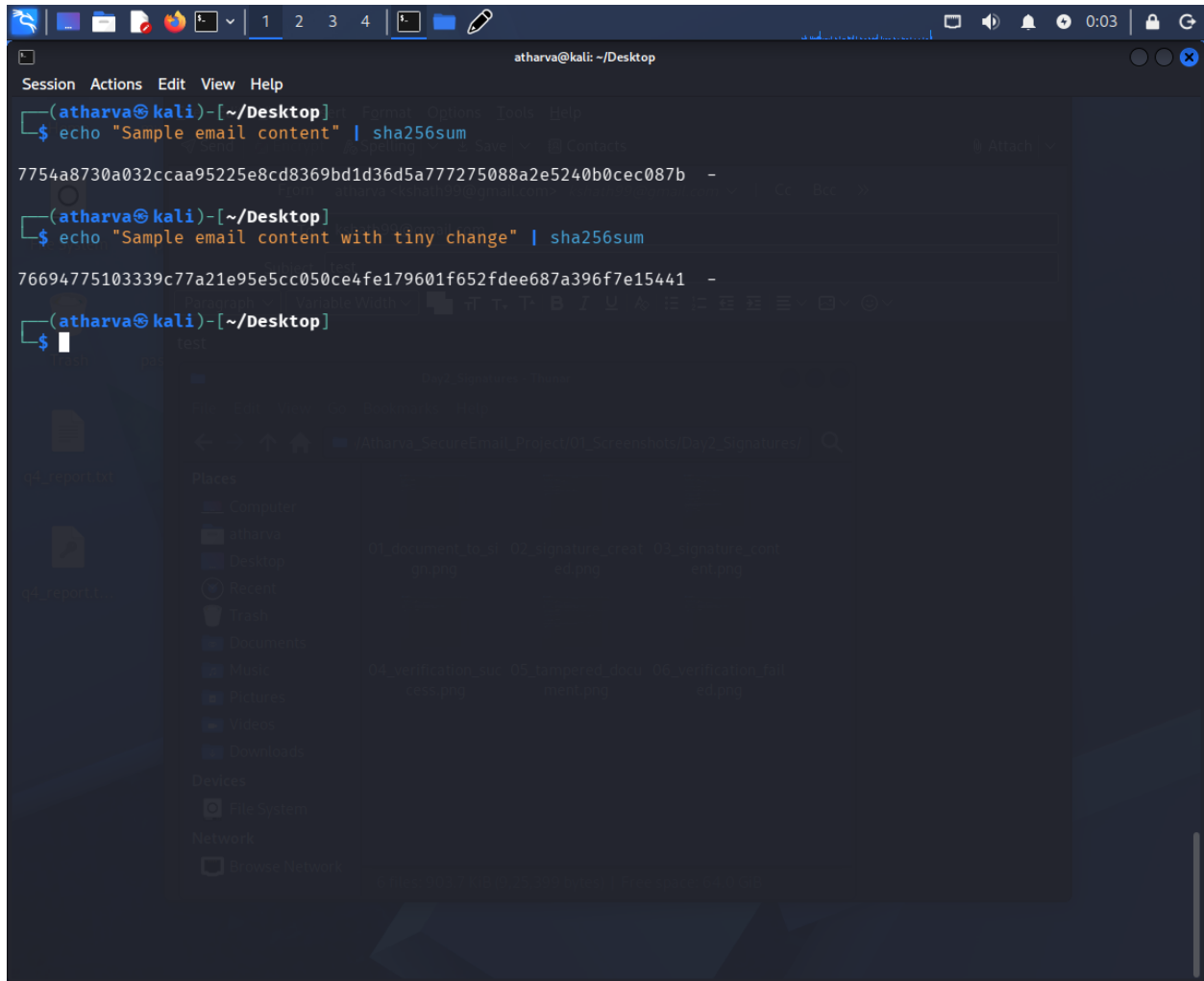
5. Hashing Algorithms

Digital signatures rely on cryptographic hash functions. A hash is a fixed-length fingerprint of arbitrary input data. The properties that make hashing useful here are:

- Same input always produces the same hash
- Any change in input - even a single bit - produces a completely different hash
- It is computationally infeasible to find two inputs that produce the same hash (collision resistance)
- The original input cannot be reconstructed from the hash (one-way property)

5.1 SHA-256 Demonstration

To make this concrete, I hashed two nearly-identical strings and compared the outputs. The dramatic difference in hashes is what gives digital signatures their tamper-detection power.



5.2 Algorithm Recommendations

Algorithm	Status	Recommendation
MD5	Broken	Do not use - collisions found
SHA-1	Deprecated	Phasing out by 2026
SHA-256	Secure	Recommended default
SHA-3	Secure	Future-proof choice

6. Signature Workflow

The complete signing and verification workflow from sender to recipient:

Sender Side

- Compose the email message
- Compute SHA-256 hash of message content
- Encrypt the hash with sender's private key (this is the signature)
- Attach the signature to the outgoing message

Recipient Side

- Receive the email and its attached signature
- Compute SHA-256 hash of received message content
- Decrypt the signature using sender's public key, revealing the original hash
- Compare both hashes - if equal, message is authentic and unchanged

7. Recommendations

- All emails carrying confidential client data must be digitally signed at the source
- Auto-signing should be enabled in Thunderbird for relevant accounts
- Private keys must be protected with strong passphrases and never stored on shared drives
- Key rotation every 12-24 months, with smooth handover during the transition window
- Revocation certificates should be generated at the time of key creation and stored offline

8. Conclusion

Digital signatures, backed by strong hashing, give the consulting firm a reliable way to detect any tampering with sensitive email content and to prove the identity of the sender. The practical demonstration on Kali shows that the workflow is straightforward and reliable: legitimate signatures verify cleanly, tampered messages fail loudly. Combined with the encryption and authentication controls in Deliverable 2, this completes the integrity and tamper-proofing layer of the secure email architecture.